

Where is the Data?

Why you Cannot Debate CPU vs. GPU Performance Without the Answer

Chris Gregg and Kim Hazelwood
University of Virginia
Computer Engineering Labs



GPUs and Data Transfer

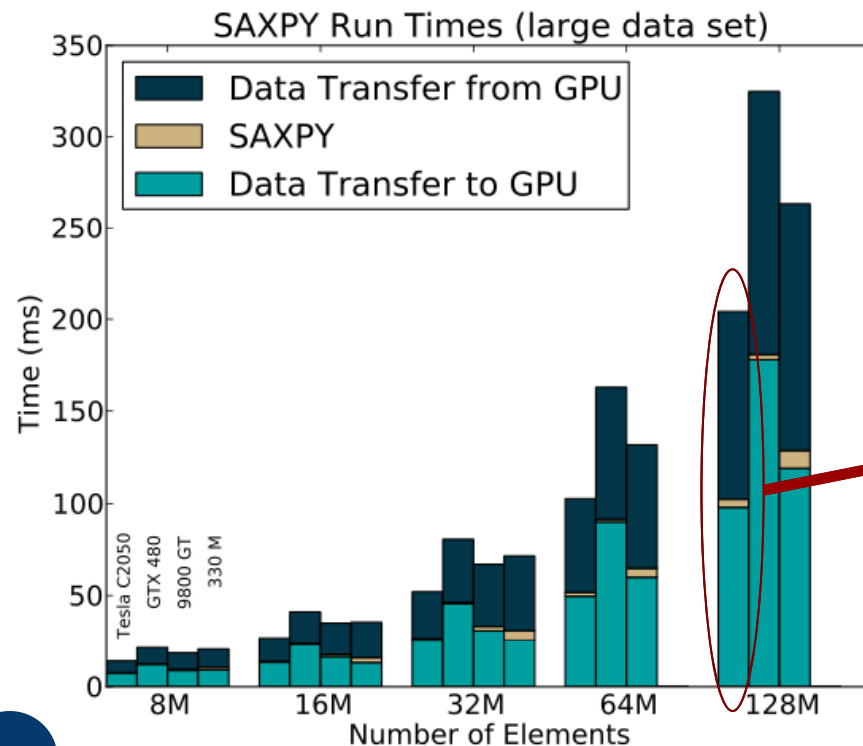
- **GPU computing offers tremendous speedup in computation. They are fast, but data transfer is a bottleneck.**
- **All data that goes to the GPU must get there through the PCI Express Bus.**
 - **Data generated *on* the GPU only sometimes has to come back to the CPU.**
 - **Some algorithms either generate or consume data, and only have one-way transfers.**

GPUs and Data Transfer

- **“GPU Speedup” is misleading without describing data transfer necessities.**
 - **Example: Sorting can be extremely fast on the GPU, but**
 - **How did the original data get to the device**
 - **What happens after it is sorted?**
- **GPUs have one I/O path: the PCIe Express bus.**

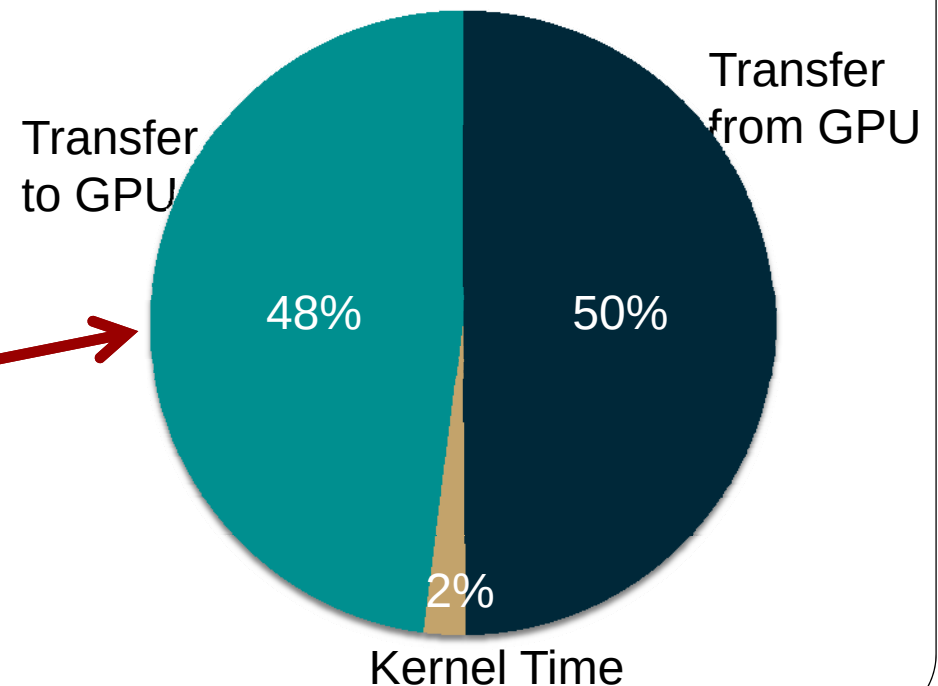
Why Data Transfer Overhead Matters

- GPU kernels are *fast*. Data transfer time can overwhelm kernel runtime.



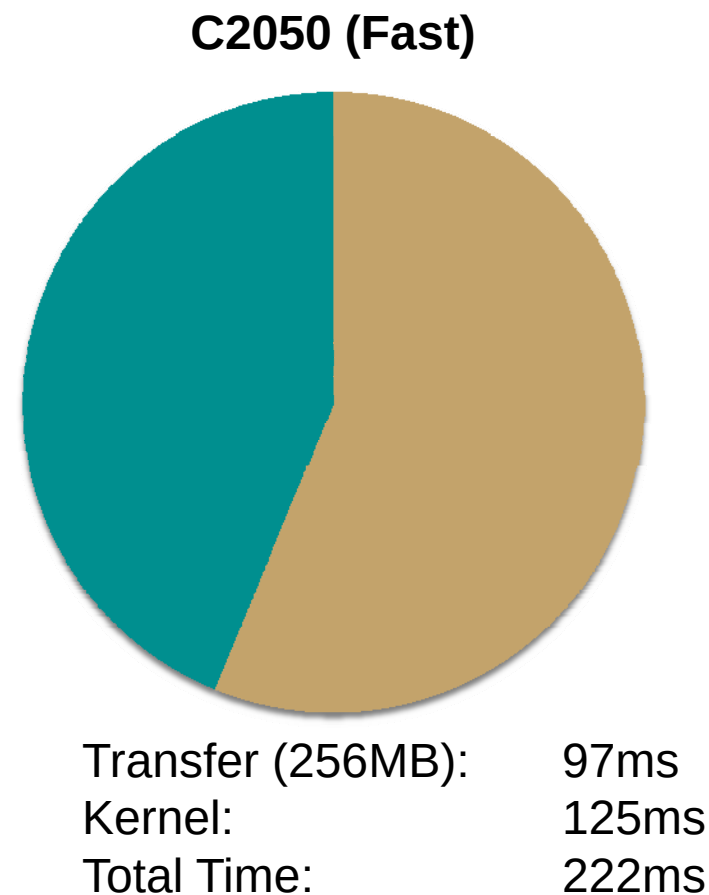
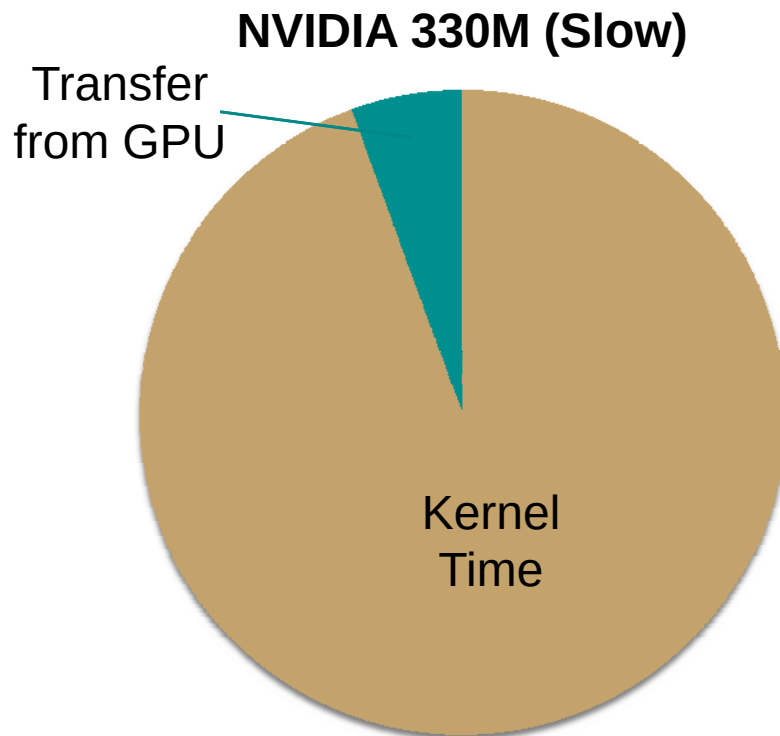
Kernel Runtime: 4ms
Data Transfer: 198ms

SAXPY (128MB)



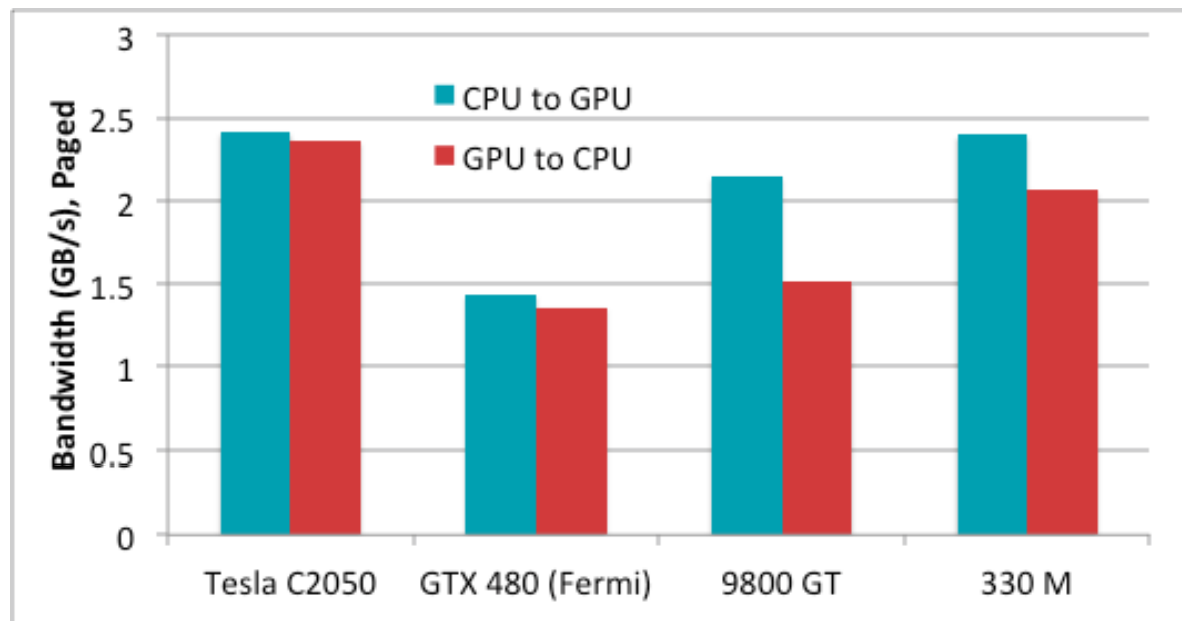
Faster GPUs Exacerbate the Problem

Radix Sort



PCI Express (PCIe) Overview

- CPUs and discrete GPUs contain separate memory systems, and memory transfer between the two happens across the PCIe bus.
- PCIe 2.1 with 16 lanes has a maximum throughput of 8GB/s, but in practice it is lower:



Dependencies:

- Paged/Pinned
- Lanes available in slot

PCI Express (PCIe) Overview

- **GPUs can move data via pinned or paged transfers, synchronously or asynchronously**
 - **Pinned transfers provide direct mapped access to CPU main memory from the GPU, however the data *still has to travel across the PCIe bus*.**
 - **Asynchronous transfers can hide memory transfer overhead**
 - **Not all algorithms support starting on partial data**
 - **Not as straightforward to program**

Categorizing Transfer Requirements

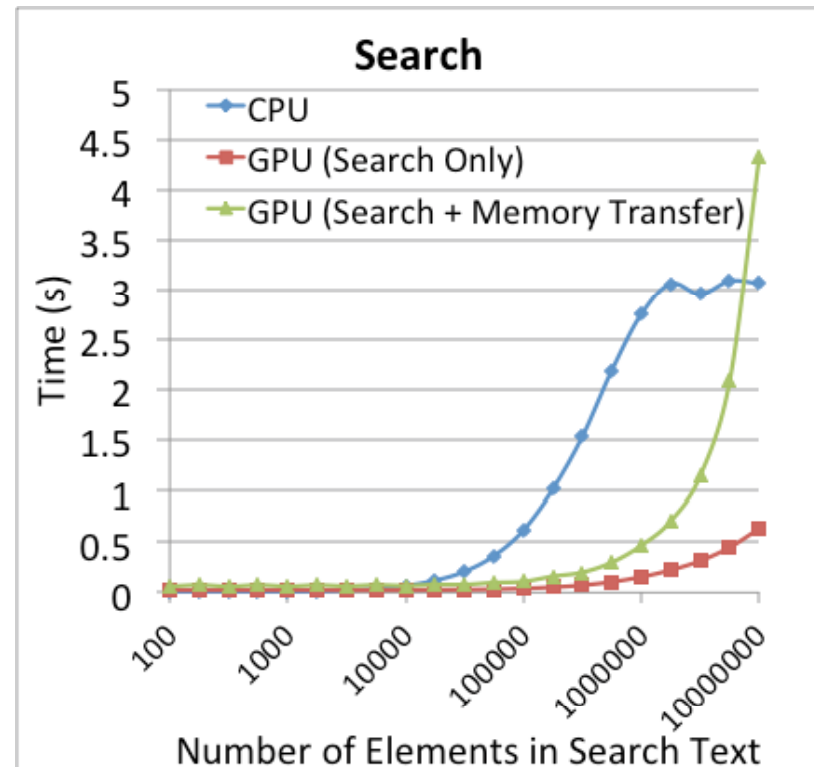
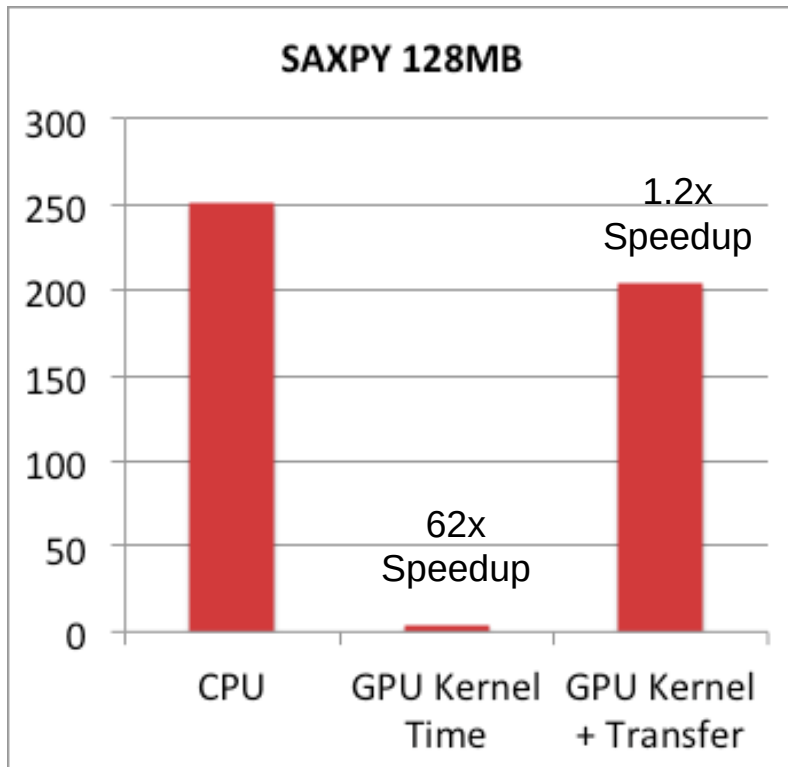
All kernels can be placed into one of the following categories, although a kernel's category could change depending on the application:

- 1. Non-Dependent (ND)**
- 2. Dependent-Streaming (DS)**
- 3. Single-Dependent-Host-to-Device (SDH2D)**
- 4. Single-Dependent-Device-to-Host (SDD2H)**
- 5. Dual-Dependent (DD)**

Categorizing Transfer Requirements

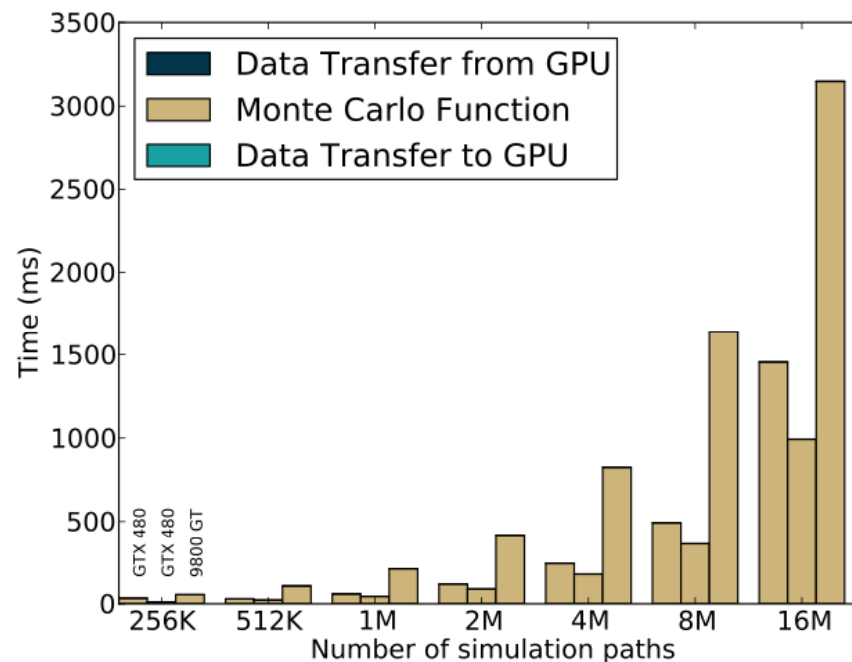
Why is categorization important?

- CPU / GPU comparisons



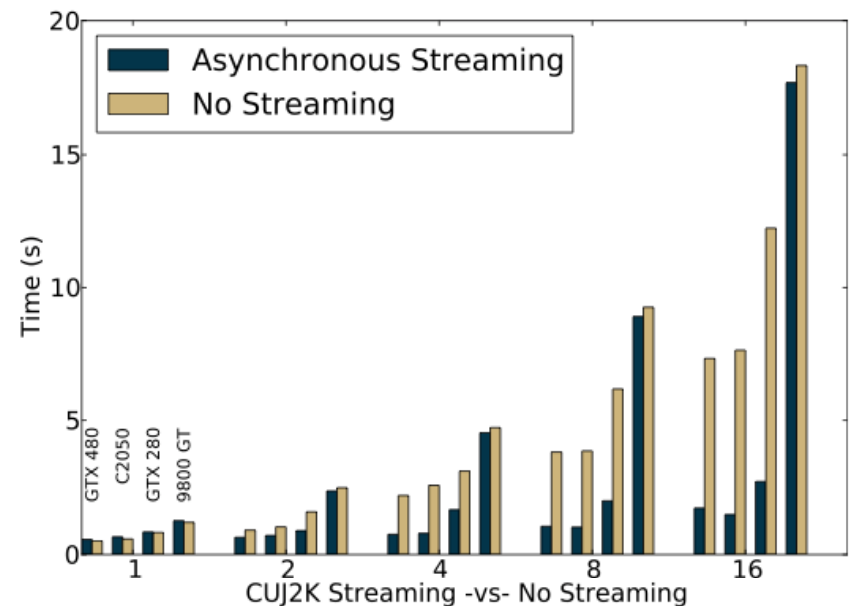
Data Transfer Categories

- **Non-Dependent (ND)**
 - Data is present on the GPU at kernel launch, and little or no data needs to be returned to CPU
 - Examples: Monte Carlo, Sort (sometimes)



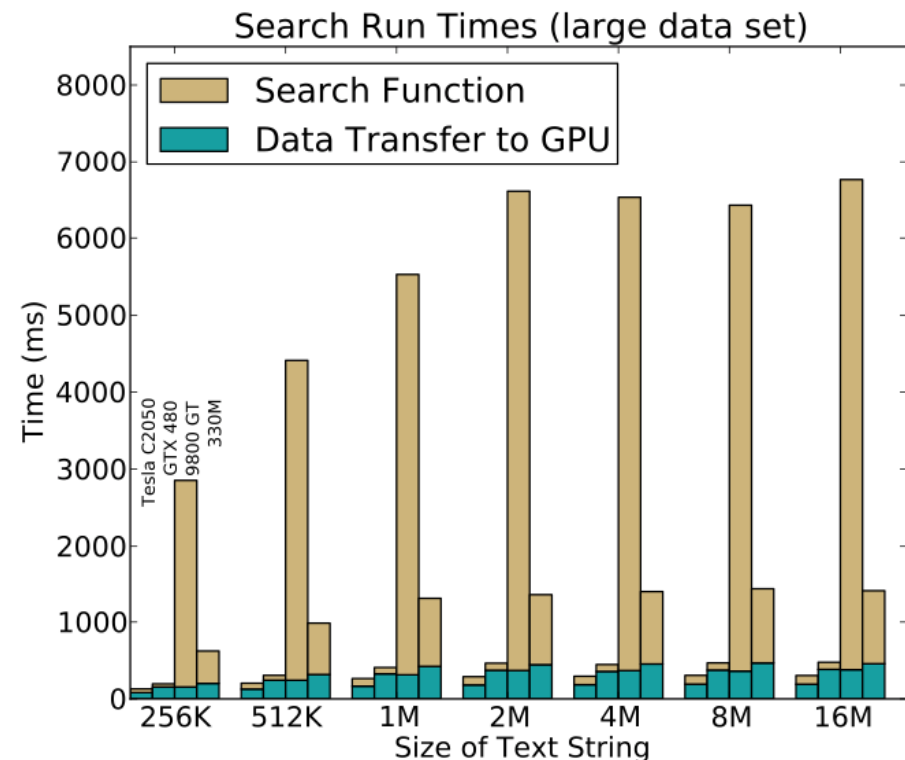
Data Transfer Categories

- **Dependent-Streaming (DS)**
 - Information is processed on the GPU and concurrently streamed over the PCI Express Bus.
 - Excellent for hiding data transfer overhead
 - More difficult to program
 - Example: JPEG Conversion



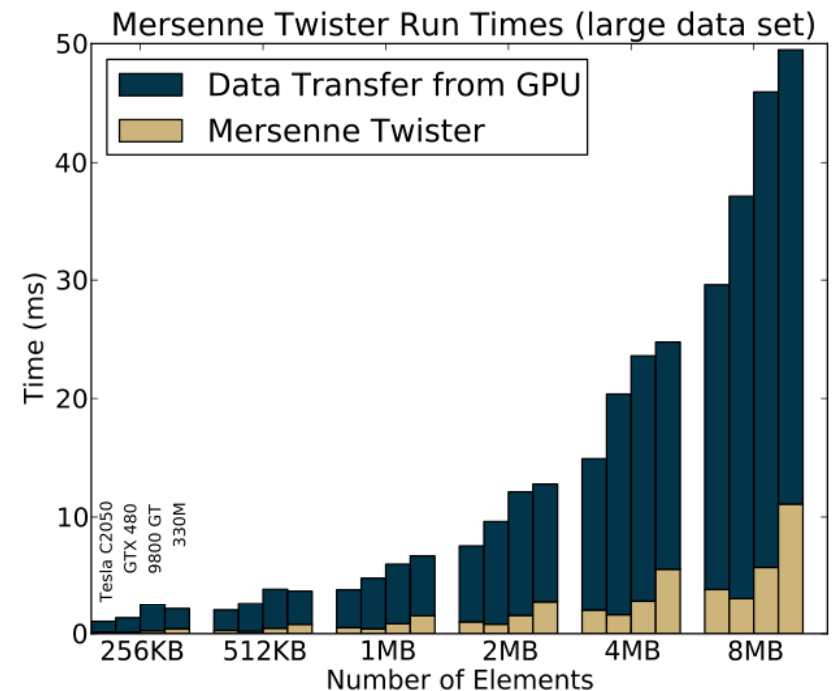
Data Transfer Categories

- **Single-Dependent-Host-to-Device (SDH2D)**
 - Data is sent to the GPU and consumed; little or no data is returned.
 - “First Kernel” Syndrome
 - Examples: Search, Histogram



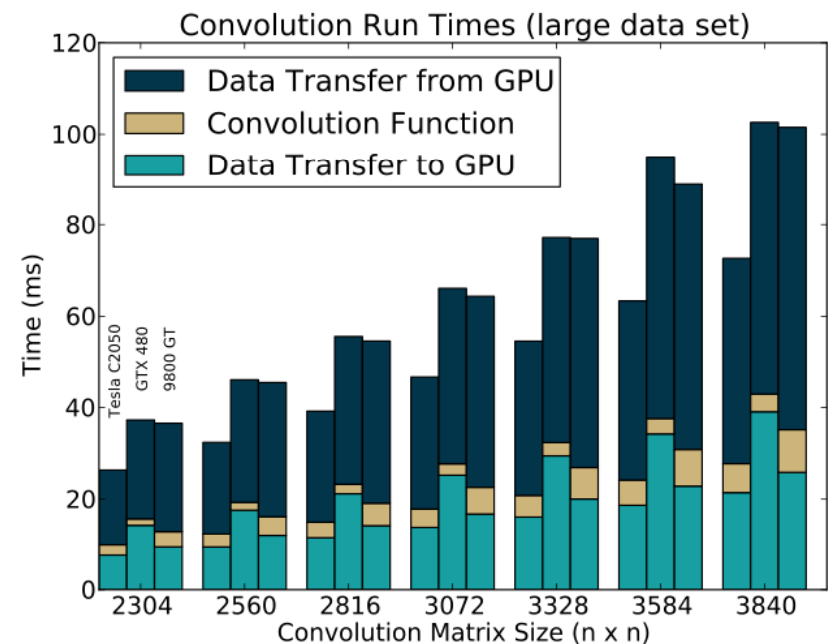
Data Transfer Categories

- **Single-Dependent-Device-to-Host (SDD2H)**
 - Data is created (or exists already) on the GPU and is returned to the CPU
 - “Last Kernel” Syndrome
 - Example: Mersenne Twister



Data Transfer Categories

- **Dual Dependent (DD)**
 - Data is sent to the GPU, is processed, and is sent back to the CPU.
 - Common for single-kernel applications
 - Examples: Matrix Multiply, Convolution, FFT.



Categorizing Transfer Requirements

Kernels can change categorization depending on their role in an application.

Example 1:

- Kernel pipelines: data is moved onto the GPU for the first kernel, manipulated in-place for other kernels, and moved back after the last kernel.**

SDH2D		ND	ND	SDD2H	
Data Transfer to GPU	Kernel 1	Kernel 2	Kernel 3	Kernel 4	Data Transfer from GPU

Changing Categories

Example 2:

- **Some kernels, such as CUJ2K, stream (DS category) when more memory is available, but don't stream in limited-memory cases (DD category).**

When reporting on speedup, “common-use case” categories should be described.

- **E.g., FFT is Dual-Dependent in general, but a common-use case would be as the last kernel in a pipeline of kernels, which would make the kernel SDD2H.**

Scheduling and Runtime Prediction

- **Heterogeneous scheduling depends on overall time to complete work, not simply kernel time.**
- **Data location is important for future scheduling decisions**
 - **We want to minimize data transfer, but maximize device utilization – this is a difficult problem when considering the data transfer overhead.**
- **Categorizing data transfer requirements for kernels helps to inform the scheduling decision.**

Conclusions

- **The CPU -vs- GPU performance question is not simply “this algorithm runs X times faster on the GPU than the CPU.”**
- **Categorizing kernel behavior by including data transfer requirements provides necessary information to the discussion.**
- **Heterogeneous scheduling can benefit greatly from the increased information.**

Questions?



SCHOOL OF ENGINEERING
AND APPLIED SCIENCE

COMPUTER ENGINEERING LABS